

소프트웨어 프로젝트 · 학습 노트

BookMate 코드 질문 노트

코드를 직접 입력하며 Claude에게 물어본 질문과 답변 모음

질문 25개 · 2026년 9월 ~ 10월

대상 코드: models.py · validator.py · store.py · manager.py

이 노트에 대하여

BookMate 코드를 공부하면서 Claude에게 코드에 대해 물어본 질문과 답변을 파일 순서대로 모았습니다. 질문은 원래 표현을 살리되 읽기 쉽게 다듬었고, 답변은 핵심 설명·예시 코드·실행 결과를 남기고 줄였습니다. 대화 중 나중에 바로잡은 내용(예: `with` 가 닫힐 때 보장하는 범위)은 바로잡은 내용으로 반영했습니다.

코드 전체를 한 번에 설명해 달라고 했던 두 번의 긴 설명은, 같은 내용이 각 파일 부분에 질문 단위로 들어 있어 따로 심지 않았습니다.

목차

1부. 전체 구조와 자료 구조	3
Q1. PPT 6번이 궁금한데 위의 자료구조 파트에서 리스트, 딕셔너리 이게 관계가 머릿속...	3
Q2. 혹시 딕셔너리와 리스트로 이제 우리가 저장한다고 했는데 장점이 어떤 걸까?	4
Q3. 궁금한 게 딕셔너리도 순서 기능이 있으면 리스트는 괜히 있는 거 아니니?	5
2부. models.py	6
Q4. from dataclasses import dataclass, fields 우선 여...	6
Q5. 그럼 dataclasses는 기본 라이브러리이고 dataclass와 fields는 ...	6
Q6. @dataclass는 세 가지를 자동으로 만들어 준다는데(__init__, __re...	7
Q7. 그럼 __init__ 같은 경우 메서드이며 처음은 무조건 self가 붙어 있으며 항...	9
Q8. 그럼 객체가 생성될 때 미리 부르는 구조여서 __init__ 안에 print가 있으...	10
Q9. GENRES={"소설", "인문", "과학", "역사", "자기계발", "기타"} ...	10
Q10. @dataclass 이 코드 설명 부탁해.	11
Q11. 그럼 if문인데 특이하게 ""처럼 문자열 형식으로 적어도 파이썬이 인식하는 거야? ...	12
Q12. @classmethod 여기 부분에 대해 설명해줘.	13
Q13. @classmethod는 @dataclass와 뭐가 다르니? 매개변수로 Book 클...	14
3부. validator.py	16
Q14. import re 여기에 대해 설명해줘.	16
Q15. class Validator: 여기에 대해 설명해줘.	17
Q16. data: 화면에서 모은 입력값 딕셔너리, 값이 전부 글자로 들어옴 — 이 말을 정...	18
4부. store.py	20
Q17. import csv, os, shutil 여기에 대해 알려줘.	20
Q18. class StoreError(Exception): ... 여기에 대해 알려줘.	21
Q19. @property를 어떻게 사용하는지 예시랑 원리를 좀 더 설명해 주고 코드에서의 ...	22
Q20. 임시 파일은 파일을 생성하기 전에 만든다고 이해했는데, 백업 경로는 어디서 사용되는...	24
5부. manager.py	25
Q21. manager.py 전체(Result, Stats, _normalize, BookM...	25
6부. 파일 처리와 안전 저장	27
Q22. 잘못된 csv 파일을 넣으면 그냥 아예 나오지 않는 거니?	27
Q23. (저장 실패 캡처와 save() 코드를 보여 주며) 이 이미지면 되니? 그리고 내가...	27
Q24. (앞의 설명을 붙여 넣으며) 여기에 대해 좀 더 자세하게 설명해 줘.	28
Q25. (save() 코드를 다시 붙여 넣으며) 좀 쉽게 설명해 줘.	29

1부. 전체 구조와 자료 구조

Q1 · 나의 질문

PPT 6번이 궁금한데 위의 자료구조 파트에서 리스트, 딕셔너리 이게 관계가 머릿속에서 잘 안 그려지는데 어떻게 자료구조가 되어 있는지 더 자세히 설명해줘.

Claude 답변

핵심: 책은 한 권, 가리키는 목록이 두 개

도서관에 비유하면 이렇습니다.

코드	도서관 비유
Book 객체	서가에 꽂힌 실제 책 (한 권에 한 권뿐)
<code>books</code> 리스트	들어온 순서대로 적은 등록 장부
<code>isbn_map</code> 딕셔너리	ISBN으로 찾는 색인 카드함

장부와 색인 카드 어디에도 책 자체는 들어 있지 않습니다. 둘 다 **책이 있는 위치(참조)**만 적혀 있고, 그 위치가 같은 책을 가리킵니다. 그래서 색인 카드로 찾아가 책에 ★ 스티커를 붙이면, 장부로 찾아가 사람도 스티커 붙은 책을 봅니다.

```
books = [ ●, ●, ● ]
          | | |
          ▼ ▼ ▼
        Book Book Book    ← 실제 객체는 한 벌
          ▲ ▲ ▲
          | | |
isbn_map = {"978..":●, "978..":●, "978..":●}
```

실제로 같은 객체인지 확인

`id()` 는 객체의 고유 번호(메모리 위치)를 돌려줍니다.

```
books[0] 의 id      : 0x7f9cab34a510
isbn_map[k] 의 id   : 0x7f9cab34a510    ← 번호가 완전히 같음
같은 객체인가(is)  : True
```

이 구조는 `add_book()` 의 세 줄에서 만들어집니다.

```
book = self._make_book(data)    # ① 객체를 딱 한 번 만든다
self.books.append(book)         # ② 리스트에 "그 객체"를 넣는다
self.isbn_map[book.isbn] = book # ③ 딕셔너리에도 "그 객체"를 넣는다
```

`append()` 나 `d[k] = v` 는 객체를 복사하지 않고 **참조만 저장**합니다.

왜 두 개를 두나

하고 싶은 일	리스트 <code>books</code>	딕셔너리 <code>isbn_map</code>
등록 순서대로 보여 주기·저장하기	잘함	—
ISBN으로 책 하나 찾기	하나씩 비교 (최대 n번)	한 번에 찾음
ISBN 중복 검사	전부 훑어야 함	<code>in</code> 으로 한 번에

메서드	쓰는 자료구조
<code>list_all()</code> , <code>search()</code> , <code>get_stats()</code> , <code>save()</code>	<code>books</code> (순서대로 훑기)
<code>update_book()</code> , <code>toggle_favorite()</code>	<code>isbn_map</code> (찾아서 객체 내용 수정 → <code>books</code> 에도 저절로 반영)
<code>add_book()</code> , <code>delete_book()</code>	둘 다 (책이 생기거나 사라지므로 함께 고쳐야 함)

마지막 줄이 이 구조의 **대가**입니다. 책이 들어오거나 빠질 때는 두 목록을 함께 고쳐야 해서, 그 일은 `add_book()` 과 `delete_book()` 에서만 하도록 제한했습니다.

삭제하면 리스트 번호는 당겨지지만(데미안이 `books[2]` → `books[1]`) ISBN 키는 그대로입니다. 그래서 화면 표에 행 번호가 아니라 ISBN을 숨겨 두고 책을 구분합니다.

만약 복사본이었다면

`isbn_map` 에 복사본을 넣으면 `==` 는 True(내용 같음)지만 `is` 는 False(다른 객체)입니다. 이때 딕셔너리로 ★를 바꾸면 복사본에만 별이 붙고, 저장에 쓰는 `books` 의 원본은 그대로라 **저장하면 별이 사라지는 버그**가 생깁니다.

책 참고 책 6장 앞부분(변수는 이름표, 별칭, `==` 와 `is`), 같은 장의 얇은 복사, 3장의 해시 테이블

Q2 · 나의 질문

혹시 딕셔너리와 리스트로 이제 우리가 저장한다고 했는데 장점이 어떤 걸까?

Claude 답변

먼저: **books.csv 파일 안에는 리스트도 딕셔너리도 없습니다.** 파일에는 글자 줄만 있고, 리스트와 딕셔너리는 프로그램이 켜져 있는 동안 **메모리에서** 책을 들고 있는 방식입니다.

`books.csv` (글자) —load()→ `books`(리스트) + `isbn_map`(딕셔너리) —save()→ `books.csv` (글자)

- **리스트의 장점: 순서.** 등록 순서대로 화면에 보여 주고, 같은 순서로 저장하고, 검색·통계처럼 전체를 훑는 일에 씁니다.
- **딕셔너리의 장점: 빨리 찾기.** 1,000권 중 마지막 책을 ISBN으로 찾아보면

리스트로 찾기: 17.4 마이크로초 (앞에서부터 하나씩 비교)
딕셔너리로 찾기: 0.04 마이크로초 (바로 찾음)

약 400배 차이입니다. 수정·삭제·즐거찾기할 책을 찾을 때, 등록 시 ISBN 중복을 확인할 때 씁니다.

- **함께 쓸 때의 장점:** 같은 책 한 권을 양쪽에서 가리키므로, 딕셔너리로 찾아 바꾸면 리스트에서도 바뀌어 있습니다. 두 곳을 따로 맞춤 필요가 없습니다.

궁금한 게 딕셔너리도 순서 기능이 있으면 리스트는 괜히 있는 거 아니니?

Claude 답변

솔직한 답은 "네, 딕셔너리 하나로도 만들 수 있다" 입니다.

```
넣은 순서 유지:  ['코스모스', '사피엔스', '데미안']
삭제·추가 후   :  ['코스모스', '데미안', '아몬드']
m[0]           → KeyError (번호로 꺼낼 수 없음)
```

파이썬 3.7부터 딕셔너리는 넣은 순서를 기억합니다. 그러니 리스트는 꼭 필요해서라기보다 **설계상 선택**입니다.

그래도 리스트를 둔 이유

1. **번호로 꺼낼 수 있음:** `books[0]` 처럼 바로 꺼내고, `books.sort()` 로 그 자리에서 정렬할 수 있습니다. 딕셔너리는 `list(m.values())[0]` 처럼 바꿔야 합니다.
2. **역할이 코드에 드러남:** "순서 있는 목록은 `books`, ISBN 색인은 `isbn_map`"으로 이름부터 나뉩니다.
3. **설계서에 그렇게 정했음:** 설계서 5.1에 두 개를 쓰기로 적었습니다.

대가

리스트 + 딕셔너리 (지금)		딕셔너리만
등록·삭제	두 곳을 함께 고쳐야 함	한 곳만
어긋날 위험	한쪽만 고치면 생김	없음
삭제 속도	리스트에서 찾아 지워 느낌	바로 지움

발표에서 이 질문이 나오면 "딕셔너리만으로도 되지만 이런 이유로 둘 다 썼고, 대가는 이렇게 관리했다"고 답하면 됩니다.

2부. models.py

Q4 · 나의 질문

```
from dataclasses import dataclass, fields
```

우선 여기에 대해 설명해줘.

Claude 답변

"파이썬에 기본으로 들어 있는 도구 두 개를 가져오겠다" 는 뜻입니다.

```
from dataclasses import dataclass, fields
#      ^^^^^^^^^^^^^      ^^^^^^^^^      ^^^^^^
#      어디서(모듈)        무엇을 가져올지
```

- `dataclasses`: 모듈 이름. 표준 라이브러리라 설치 필요 없음
- `dataclass`, `fields`: 그 모듈 안의 도구 두 개

`import dataclasses` 로 전체를 가져오면 `dataclasses.dataclass` 처럼 길게 써야 하지만, `from ... import` 로 가져오면 `dataclass` 만 쓰면 됩니다. 모듈은 `dataclasses` (s 있음), 가져오는 도구는 `dataclass` (s 없음) 입니다.

도구	쓰이는 곳	하는 일
<code>dataclass</code>	<code>class Book</code> 위의 <code>@dataclass</code>	필드만 적으면 생성자 같은 기본 코드를 자동으로 만들어 줌
<code>fields</code>	<code>FIELD_NAMES = ...</code>	Book에 어떤 필드가 있는지 목록으로 알려 줌

파이썬에서 `dataclass`, `fields` 를 쳐 보면 둘 다 `<function ...>` 이 나옵니다. 둘 다 함수입니다.

Q5 · 나의 질문

그럼 `dataclasses`는 기본 라이브러리이고 `dataclass`와 `fields`는 모듈인 거지? 또 `dataclass`와 `fields`에 대해서 좀 더 구체적으로 설명해줘.

Claude 답변

용어 정리

```
표준 라이브러리 파이썬에 기본으로 들어 있는 모듈들의 모음
└─ dataclasses      ← 모듈 (파일 하나: ../dataclasses.py)
    │ dataclass     ← 함수
    └─ fields       ← 함수
```

모듈은 파이썬 코드가 든 파일 하나입니다. 우리가 만든 `models.py` 도 모듈입니다. `dataclass` 와 `fields` 는 그 안의 함수입니다.

dataclass: 클래스의 기본 코드를 대신 만들어 주는 함수

```
class 일반책:                                # 직접 작성
    def __init__(self, title, year):
        self.title = title
        self.year = year

@dataclass
class 데이터책:                             # @dataclass 사용
    title: str
    year: int = 2000
```

해 본 것	일반책	데이터책
<code>print(책)</code>	<code><main.일반책 object at 0x...></code>	<code>데이터책(title='코스모스', year=2006)</code>
내용 같은 두 책 <code>==</code>	<code>False</code>	<code>True</code>
<code>데이터책("데미안")</code>	—	<code>year=2000</code> 기본값이 들어감

`@dataclass` 는 `__init__` (생성자), `__repr__` (print 모양), `__eq__` (내용 비교)를 자동으로 만들어 줍니다. 일반 클래스의 `==` 가 `False`인 건, 따로 정하지 않으면 "같은 객체인가"로 비교하기 때문입니다.

`@dataclass` 는 `데이터책 = dataclass(데이터책)` 을 짧게 쓴 것입니다.

fields: 데이터 클래스의 필드 목록을 알려 주는 함수

이름 (name)	타입 (type)	기본값 (default)
title	<class 'str'>	MISSING ← 기본값 없음 표시
year	<class 'int'>	2000

우리 코드는 이름만 뽑아 CSV 헤더(`FIELD_NAMES`)로 씁니다. `fields` 는 `@dataclass` 가 붙은 클래스에만 쓸 수 있습니다.

책 참고 책 5장 앞부분

Q6 · 나의 질문

`@dataclass`는 세 가지를 자동으로 만들어 준다는데(`__init__`, `__repr__`, `__eq__`) 여기에 대해서 몰라서 애매해. 그리고 `@`형식으로 적으면 왜 같은 내용의 객체가 같다고 되니? 또 필드가 9개면 `__init__`을 9줄 써야 한다는데 `__init__`이 어떤 건지 알려줘.

Claude 답변

0. `__이름__` 메서드: 파이썬이 대신 불러 주는 메서드

우리가 쓰는 문법	파이썬이 몰래 부르는 메서드
<code>Book("코스모스", 2006)</code>	<code>__init__</code>
<code>print(book)</code>	<code>__repr__</code>
<code>book1 == book2</code>	<code>__eq__</code>

이름으로 직접 부르지 않고, 특정 문법을 쓰면 파이썬이 알아서 부릅니다. 그래서 **특별 메서드**라고 합니다.

1. `__init__`: 객체를 만들 때 값을 넣어 주는 메서드

```
class 책:
    def __init__(self, title, year):
        print("__init__이 불렸습니다")
        self.title = title
        self.year = year

b = 책("코스모스", 2006)
print(b.title)
```

`책("코스모스", 2006)` 을 쓰면 파이썬이 ① 빈 객체를 만들고 ② `__init__` (그 객체, "코스모스", 2006) 을 부르고 ③ 채워진 객체를 돌려줍니다. `self` 는 ①에서 만든 바로 그 객체입니다.

필드가 8개인 Book을 직접 쓰면 `def` 줄 1개 + `self.∞ = ∞` 8줄 = 9줄입니다. `@dataclass` 는 이 반복을 대신 써 줍니다.

2. `__repr__`: print 했을 때 보여 줄 글자

```
def __repr__(self):
    return f"책(title={self.title!r}, year={self.year})"
```

없으면 `<main.책 object at 0x...>`, 있으면 `책(title='코스모스', year=2006)` 이 나옵니다. (엄밀히는 `print` 는 `__str__` 을 먼저 찾고 없으면 `__repr__` 을 씁니다.)

3. `__eq__`: `==` 로 비교할 때의 기준

```
def __eq__(self, other):
    return self.title == other.title and self.year == other.year
```

`a == c` 를 쓰면 파이썬이 `a.eq(c)` 를 부릅니다. 없으면 기본 기준이 "같은 객체인가"라서, 내용이 같아도 따로 만든 두 객체는 `False`입니다.

4. 그래서 `@dataclass` 를 붙이면 왜 같다고 나오나

`@dataclass` 가 위의 세 메서드를 대신 써서 클래스에 넣어 주기 때문입니다. 그중 `__eq__` 이 필드를 전부 비교하도록 만들어지니 내용이 같으면 같다고 나옵니다. `@` 가 마법을 부리는 게 아니라 `@` 로 부른 함수가 비교 코드를 써 준 것입니다.

책 참고 책 1장(특별 메서드), 5장

그럼 `__init__` 같은 경우 메서드이며 처음은 무조건 `self`가 붙어 있으며 항상 실행이 된다고 이해했어. 그런데 `print(book)` 했을 때 `__repr__`이 작동한다고 했는데 `b.title`과는 다른 걸이니? 이름은 꼭 `init`, `repr`, `eq` 같은 형태여야 하니? `b.title`을 부를 때 자동으로 앞의 `print("__init__ ...")`가 불러서 출력된 거니? `__repr__`은 `fields`와 비슷한 형태로 출력되는데, `@dataclass`로 매개변수 지정 → 객체 생성 → `print`를 해도 그렇게 나오니? `@dataclass`는 `title: str`처럼 이름과 타입만 정하면 되는 거니?

Claude 답변

1. "메서드이고, self가 붙고, 항상 실행된다"

앞의 두 개는 맞습니다. 세 번째는 "객체를 만들 때 딱 한 번 실행된다"가 정확합니다. 객체를 쓸 때마다 실행되지 않습니다. 첫 매개변수 이름 `self`는 문법이 아니라 모든 개발자가 지키는 약속입니다.

2. b.title 과 print(b) 는 다른 걸인가

코드	일어나는 일	특별 메서드
<code>b = 책("코스모스", 2006)</code>	객체를 만들고 값을 넣음	<code>__init__</code>
<code>b.title</code>	넣어 둔 값을 꺼내기만 함	안 불림
<code>print(b)</code>	객체 전체를 글자로 보여 달라고 함	<code>__repr__</code>

3. b.title 을 부를 때 print가 같이 불린 건가

아닙니다.

```

객체 만들기 직전
__init__이 불렸습니다      ← 책("코스모스", 2006) 줄에서 찍힘
객체 만들기 끝
코스모스                  ← b.title
코스모스                  ← b.title 한 번 더 (메시지 다시 안 찍힘)
  
```

4. 이름은 꼭 그대로여야 하나

네, 글자 하나까지 같아야 합니다. `__init__` (밑줄 하나)로 쓰면 `TypeError: 오타책() takes no arguments`가 납니다. `__repr__`을 `__rper__`처럼 틀리면 에러 없이 무시되니 더 조심해야 합니다.

5. @dataclass 로 만들고 print해도 그 형식이 나오나

네. 다만 `fields`와 `print`는 보는 대상이 다릅니다.

	보여 주는 것	비유
<code>fields(데이터책)</code>	클래스에 어떤 칸이 있고 타입이 뭔지	빈 양식지
<code>print(x)</code>	객체 하나에 실제로 든 값	작성된 서류

6. 이름: 타입 만 적으면 되나

맞습니다. 적은 순서가 `__init__`의 매개변수 순서가 되고 `= 값`은 기본값입니다. 규칙 세 가지:

- 타입은 강제되지 않음: `데이터책(123, "이천육")`도 그대로 들어감

2. **: 타입을 빼면 필드가 아님:** `year = 2000` 처럼 쓰면 필드 목록에서 빠짐
3. **기본값 있는 필드는 뒤에:** 앞에 두면 `non-default argument 'title' follows default argument`

Q8 · 나의 질문

그럼 객체가 생성될 때 미리 부르는 구조여서 `__init__` 안에 `print`가 있으면 자동으로 부르는 구조인 거구나. `__repr__`을 통해 객체를 `print`하면 `fields` 형태로 출력되는데, 이건 `dataclasses` 모듈에서 `fields`를 불러와서 그런 거야? 아니면 `dataclass`가 그 역할을 하는 거야?

Claude 답변

앞의 이해는 맞습니다.

`print` 형식은 `fields`를 가져와서가 아니라 `@dataclass`가 `__repr__`을 만들어 줘서 생깁니다.

실험	결과
<code>dataclass</code> 만 가져오고 <code>fields</code> 는 안 가져옴	<code>책A(title='코스모스', year=2006)</code> → 잘 나옴
<code>fields</code> 만 가져오고 <code>@dataclass</code> 는 안 붙임	<code><main.책B object at 0x...></code> → 안 나옴

비슷해 보이는 이유: `@dataclass`는 클래스를 고칠 때 **필드 목록을 클래스 안에 적어 줍니다**(`['title', 'year']`). `__repr__`도 `fields()`도 **같은 목록을 읽어서 쓰기** 때문입니다.

Q9 · 나의 질문

```
GENRES={"소설", "인문", "과학", "역사", "자기계발", "기타"}
_TRUE={"true", "1", "o", "y", "yes"}
_FALSE={"false", "0", "x", "n", "no", ""}
def _to_bool(text: str) -> bool: ...
```

여기에 있는 코드에 대해서 설명 부탁해.

Claude 답변

먼저 고칠 곳: GENRES의 괄호

`{}`는 **집합**, 원래 코드의 `()`는 **튜플**입니다. 집합은 순서를 기억하지 않아 실행할 때마다 순서가 바뀝니다.

```
['자기계발', '소설', '과학', '역사', '기타', '인문']
['자기계발', '소설', '인문', '기타', '과학', '역사']
```

드롭다운 순서가 바뀌고, 등록 창이 "마지막 항목 = 기타"로 가정하는 부분도 들어집니다. **순서가 중요한 목록은 튜플, "들어 있나?"만 볼 때는 집합을 씁니다.**

`_TRUE`, `_FALSE`

CSV에는 읽음 여부가 `"True"`라는 **글자**로 저장됩니다. 사용자가 엑셀에서 `true`, `1`, `0`로 고쳐도 알아들도록 인정할 글자를 모았습니다. 순서가 필요 없고 포함 여부만 보므로 집합입니다. `_FALSE`의 `""`는 빈 칸을 "안 읽음"으로 처리하기 위해서입니다. 이름 앞 `_`는 "이 파일 안에서만 쓴다"는 약속입니다.

_to_bool()

```
bool("False") → True ← 틀림!
```

비어 있지 않은 글자는 무조건 참이라 직접 비교하는 함수가 필요합니다.

```
value = text.strip().lower()      # 앞뒤 공백 제거, 소문자로 통일
if value in _TRUE: return True
elif value in _FALSE: return False
raise ValueError(f"True/False로 해석할 수 없는 값: {text!r}")
```

```
'True' → True / ' FALSE ' → False / '0' → True / '' → False
'읽음' → ValueError
```

모르는 값을 조용히 False로 바꾸지 않고 에러를 내면, store.py가 그 줄을 "형식이 잘못된 행"으로 건너뛰고 사용자에게 알려 줍니다. (**elif**로 쓴 것과 원래의 두 번째 **if**는 **return** 때문에 결과가 같습니다.)

책 참고 2장(튜플), 3장(집합)

Q10 · 나의 질문

```
@dataclass
class Book:
    title: str ... rating: float|None = None ...
    def to_row(self) -> list[str]: ...
```

이 코드 설명 부탁해.

Claude 답변

필드 8줄

그룹	필드	만들 때
기본값 없음	title, author, year, genre, isbn	반드시 넣어야 함
기본값 있음	rating, read, favorite	안 넣으면 기본값

```
Book("제목만")
# → TypeError: missing 4 required positional arguments: 'author', 'year', 'genre', and 'isbn'
```

rating: float | None = None은 "실수 또는 없음, 기본값은 없음"입니다. 0점과 "아직 안 매김"을 구분해야 평균 별점이 틀어지지 않습니다.

to_row(): 책 한 권을 CSV 한 줄로

CSV에는 글자만 저장되므로 전부 글자로 바꾼 목록을 만듭니다. 순서는 필드 순서(= CSV 헤더 순서)와 같습니다.

코드	원래 값	바뀐 글자
<code>str(self.year)</code>	2006	'2006'
<code>str(self.read)</code>	True	'True'
별점 줄	4.5 / 5 / None	'4.5' / '5.0' / ''

```
"" if self.rating is None else f"{self.rating:.1f}"
```

A if 조건 else B 형태로, 목록 안에는 if문을 못 넣어서 한 줄로 줄인 것입니다. `:.1f` 는 소수점 아래 1자리 표시라 5 도 '5.0' 이 됩니다.

책 참고 11장(`:.1f` 포맷), 8장(타입 표기)

Q11 · 나의 질문

그럼 if문인데 특이하게 ""처럼 문자열 형식으로 적어도 파이썬이 인식하는 거야? is None으로 적는다고 내부에서 크게 달라지는 건 없는 거니? 이 명령어는 문자열이 오면 그냥 오류인 거니?

Claude 답변

1. "" 가 if 앞에 와도 되나

네. if문이 아니라 조건 표현식이기 때문입니다.

	if문	조건 표현식
모양	<code>if 조건: 다음 줄에 할 일</code>	<code>값A if 조건 else 값B</code>
하는 일	코드를 실행할지 고름	두 값 중 하나를 고름

앞뒤에는 어떤 값이든 올 수 있습니다. `"짝수" if n % 2 == 0 else "홀수"`. `else` 는 반드시 있어야 합니다.

2. `is None` 과 `== None`

`==` 는 "값이 같니?"를 객체의 `__eq__` 에 묻고, `is` 는 "같은 물건이니?"를 파이썬이 직접 확인합니다. `None`은 파이썬 전체에 하나뿐이라 `is` 가 정확합니다.

```
x == None : True    ← __eq__이 항상 True라고 거짓말하는 클래스
x is None : False   ← 속지 않음
```

3. rating에 문자열이 오면

```
4.5 → '4.5'
5 → '5.0'
'4.5' → ValueError: Unknown format code 'f' for object of type 'str'
'' → ValueError (빈 글자도 None이 아님)
```

정상 사용에서는 나지 않습니다. Book을 만드는 두 입구(`from_row`, `_make_book`)가 모두 미리 `float()` 로 바꾸기 때문입니다. 타입 표시는 강제되지 않으니 입구에서 변환해 두는 것입니다.

책 참고 6장 앞부분(`is` 와 `==`)

Q12 · 나의 질문

```
@classmethod
def from_row(cls, row: dict[str, str]) -> "Book": ...
FIELD_NAMES: tuple[str, ...] = tuple(f.name for f in fields(Book))
```

여기 부분에 대해 설명해줘.

Claude 답변

먼저: 오타 두 개 (`yaer`, `raring`)

오타가 있으면 `cls(...)` 에서 `TypeError`가 나는데, 아래 `except` 가 `TypeError`를 잡아 "형식 오류 행"으로 바꿔 버려서 CSV의 모든 책이 조용히 사라집니다.

```
ValueError: 형식 오류 행: {...}
진짜 원인: Book.__init__() got an unexpected keyword argument 'yaer'
```

넓게 잡는 `except` 는 내 코드의 실수까지 숨길 수 있다는 교훈입니다. 테스트(`tests.test_store`)가 이런 실수를 잡아 줍니다.

`@classmethod` 와 `cls`

`to_row(self)` 는 있는 책에게 시키는 일, `from_row` 는 아직 책이 없는 상태에서 새 책을 만드는 일입니다. 그래서 첫 자리에 책 대신 **Book 클래스 자체(`cls`)** 를 받고, `return cls(...)` 가 곧 `Book(...)` 입니다. 반환 타입 "**Book**" 의 따옴표는 아직 클래스를 만드는 중이라 이름을 쓸 수 없어서입니다.

`or ""` 패턴

`DictReader` 는 칸이 모자란 줄을 `None` 으로 채우고, `None.strip()` 은 에러입니다. `or` 는 왼쪽이 거짓이면 오른쪽을 줍니다.

```
None or ""      → ""
"" or "기타"    → "기타"    (genre의 두 번째 or)
```

여러 예외를 `ValueError` 하나로

예외	언제
<code>KeyError</code>	열 자체가 없을 때
<code>TypeError</code>	<code>int(None)</code>
<code>AttributeError</code>	<code>None.strip()</code>

전부 `ValueError` 로 바꿔 올리면 `store.py` 는 `except ValueError` 한 줄로 처리합니다. `from exc` 는 원래 원인을 붙여 둡니다.

FIELD_NAMES

`fields(Book)` 에서 이름만 뽑아 튜플로 만든 CSV 헤더입니다. Book 정의가 끝난 뒤에야 부를 수 있어서 클래스 밖(들여쓰기 없음)에 둡니다.

책 참고 11장(대안 생성자)

Q13 · 나의 질문

@classmethod는 @dataclass와 뭐가 다르니? 매개변수로 Book 클래스 자체가 들어온다는 소리를 이해 못 했어. DictReader는 어디에 있는 거니? title·isbn·rating만 딕셔너리에서 꺼내는데 나머지는 따로 안 해도 되니? cls라는 매개변수를 지금 만들고 있는 거야? from_row는 to_row로 받은 값을 정리해 준다는 개념이야? FIELD_NAMES에서 : 뒤는 형태고 f.name과 f는 반복문에서 어떻게 쓰이니?

Claude 답변

1. @classmethod와 @dataclass

	@dataclass	@classmethod
붙이는 곳	클래스 위	메서드 위
하는 일	<code>__init__</code> 등을 만들어 클래스에 넣음	그 메서드의 첫 매개변수에 무엇이 들어올지를 바꿈

2. "Book 클래스 자체가 들어온다" — 붕어빵 틀

- `Book` (괄호 없음) = 붕어빵 틀
- `Book("코스모스", 2006)` = 틀로 찍은 붕어빵 하나

```
book.to_row()      → self = Book(title='코스모스', year=2006)  ← 붕어빵
Book.from_row(row) → cls  = <class '__main__.Book'>             ← 틀
```

3. cls를 지금 만드는 건가

아닙니다. 받아 두는 이름일 뿐이고 파이썬이 자동으로 넣습니다. `Book.from_row(row)` 는 실제로 `from_row(Book, row)` 입니다.

4. DictReader는 어디에

표준 라이브러리 `csv` 모듈 안에 있고, `store.py`의 `load()` 에서 `csv.DictReader(f)` 로 씁니다. 첫 줄을 키로 삼아 나머지 줄을 딕셔너리로 바꿔 줍니다.

5. 나머지 값은

전부 `row[...]` 로 꺼냅니다. `cls(...)` 안에서 바로 꺼낼 뿐입니다. `title·isbn`은 비었는지 먼저 검사해야 해서, `rating_text`는 두 번 쓰여서 미리 변수에 담았습니다.

6. from_row와 to_row

반대 방향의 짝입니다.

```

Book → to_row() → 글자 목록 → csv.writer → books.csv
Book ← from_row() ← 딕셔너리 ← DictReader

```

7. FIELD_NAMES

: `tuple[str, ...]` 은 타입 안내입니다. 한 줄 반복을 풀면:

```

이름들 = []
for f in fields(Book):      # 필드 정보(Field)를 하나씩 f로
    이름들.append(f.name)    # 이름만 담음
FIELD_NAMES = tuple(이름들)

```

책 참고 11장(클래스 메서드), 2장(한 줄 반복)

3부. validator.py

Q14 · 나의 질문

```
import re
from datetime import date
from typing import NamedTuple
```

여기에 대해 설명해줘.

Claude 답변

줄	모듈	가져오는 것	종류
<code>import re</code>	re	모듈 전체	모듈
<code>from datetime import date</code>	datetime	<code>date</code> 하나	클래스
<code>from typing import NamedTuple</code>	typing	<code>NamedTuple</code> 하나	클래스를 만드는 도구

re: 정규식(글자 패턴)

모듈 전체를 가져와서 `re.compile(...)` 처럼 씁니다. ISBN 검사에 쓰입니다.

'978-89-01' → True / '978abc' → False / '---' → True ← 그래서 숫자 존재를 따로 검사

date: 올해 구하기

```
date.today()      → 2026-10-01
date.today().year → 2026
```

올해를 코드에 직접 적지 않아 내년에도 고칠 필요가 없습니다.

NamedTuple: 이름 붙은 튜플

```
class FieldError(NamedTuple):
    field: str
    message: str
```

`e.field` → 'year' (이름으로) / `e[0]` → 'year' (번호로도)

	<code>@dataclass</code> (Book)	<code>NamedTuple</code> (FieldError)
만든 뒤 값 바꾸기	됨	안 됨 (AttributeError)
번호로 꺼내기	안 됨	됨

Book은 값이 계속 바뀌어야 하고, FieldError는 만들고 읽기만 하니 바뀌지 않는 NamedTuple이 맞습니다.

책 참고 5장

Q15 · 나의 질문

```
class Validator:
    REQUIRED = {...}
    def validate(self, data: dict, existing_isbns: set[str] | None = None)
    -> list[FieldError]: ...
```

여기에 대해 설명해줘.

Claude 답변

이 함수는 "입력값을 받아서, 잘못된 곳을 전부 찾아 목록으로 돌려준다" 입니다.

정상	→ 통과 (빈 목록)
전부 빈칸	→ 제목·저자·연도·ISBN 오류 4개가 한꺼번에
연도 "이천육"	→ 출판연도는 숫자로 입력하세요
ISBN "978abc"	→ ISBN은 숫자와 하이픈 (-)만 입력하세요
등록: 중복 ISBN	→ 이미 등록된 도서입니다
수정: 같은 ISBN	→ 통과
별점 "3.3"	→ 별점은 0.0~5.0 사이, 0.5 단위로 입력하세요

① 필수 항목

```
for name, message in self.REQUIRED.items():
    if not str(data.get(name, "")).strip():
        errors.append(FieldError(name, message))
missing = {e.field for e in errors}
```

`data.get(name, "")` 은 칸이 없으면 `""`, `strip()` 후 비어 있으면 거짓이라 `not` 으로 "비어 있으면"이 됩니다.
`missing` 은 이미 빈칸으로 걸린 칸 이름 집합입니다.

② 출판연도: try / except / else

칸	언제 실행	하는 일
<code>try</code>	먼저	정수로 바꿔 봄
<code>except ValueError</code>	실패했을 때만	"숫자로 입력하세요"
<code>else</code>	성공했을 때만	범위(1000~올해) 검사

`1000 <= year <= this_year` 는 연쇄 비교입니다.

③ ISBN

`fullmatch` 는 글자 전체가 규칙에 맞아야 통과(`match` 는 앞부분만 봄). `any(ch.isdigit() for ch in isbn)` 은 숫자가 하나라도 있는지 봅니다. 중복 검사는 `existing_isbns is not None` 일 때만, 즉 **등록일 때만** 합니다(수정 때는 자기 자신 때문에 막히면 안 되므로).

④ 별점

비어 있으면 통과. 변환 실패 시 일부러 `-1.0` 을 넣어 범위 검사에 걸리게 합니다. 0.5 단위는 **2를 곱해 정수인자로** 봅니다 ($3.5 \times 2 = 7.0$ 통과, $3.3 \times 2 = 6.6$ 실패).

return errors

예외를 던지지 않고 다 검사한 뒤 목록을 돌려줘서, 등록 창이 모든 오류를 한 번에 보여 줄 수 있습니다.

책 참고 18장(try/else), 3장(집합 만들기), 2장·17장(any)

Q16 · 나의 질문

data: 화면에서 모은 입력값 딕셔너리, 값이 전부 글자로 들어옴 — 이 말을 정확히 이해 못 하겠는데 data: dict니까 값들을 str로 본다는 거니? existing_isbns는 어디에 저장되어 있니? .items는 REQUIRED와 무슨 상관이니? missing에 대해, year가 빈칸이면 어떻게 되는지 알려줘. not in 문법도 더 설명해줘.

Claude 답변

1. data: dict 이면 값을 글자로 본다는 뜻인가

아닙니다. data: dict 는 안내일 뿐 값을 바꾸지 않습니다. 값이 글자인 건 data를 만드는 `form_data()` 가 입력칸의 `.text()` 로 값을 모으기 때문입니다. 사용자가 2006 을 쳐도 글자 "2006" 입니다. 다만 체크박스의 `.isChecked()` 는 진짜 True/False라 `read`, `favorite`는 글자가 아닙니다.

```
{"title": "코스모스", "year": "2006", "isbn": "978-1", "rating": "4.5", "read": True,
 "favorite": False}
```

2. existing_isbns는 어디에

Validator에는 저장돼 있지 않고 **검사할 때마다 넘겨받습니다**. manager.py의 `add_book()` 이 `set(self.isbn_map)` 으로 ISBN만 뽑아 넘깁니다. Validator는 아무것도 기억하지 않는 검사기입니다.

3. .items()와 REQUIRED

`.items()` 는 딕셔너리라면 누구나 가진 기능입니다.

```
for k in REQUIRED      → 'title', 'year'                (키만)
REQUIRED.items()     → ('title', '제목을 입력하세요'), ... (짝으로)
```

칸 이름과 메시지가 둘 다 필요해서 짝으로 받아 `name`, `message` 로 나눠 답습니다.

4. missing과 year 빈칸

① 필수 검사에서 연도가 빈칸이라 "출판연도를 입력하세요"가 들어가고 `missing = {'year'}`. ② `if "year" not in missing`: 이 거짓이라 연도 검사 전체를 건너뜁니다.

```
missing 사용   : [('year', '출판연도를 입력하세요')]
missing 미사용 : [('year', '출판연도를 입력하세요'), ('year', '출판연도는 숫자로 입력하세요')]
```

5. not in

`in`은 "왼쪽이 오른쪽 안에 있나?", `not in`은 그 반대입니다.

오른쪽	확인하는 것	예
집합·리스트·튜플	원소가 있나	<code>"year" in {"title", "year"}</code> → True
딕셔너리	키가 있나	<code>"title" in {"title": 1}</code> → True
글자	부분 글자가 있나	<code>"사피" in "사피엔스"</code> → True

`rating not in (None, "")`은 "None도 빈 글자도 아니면", 즉 뭔가 입력돼 있으면 검사한다는 뜻입니다.

4부. store.py

Q17 · 나의 질문

```
import csv, os, shutil
from pathlib import Path
from typing import NamedTuple
from .models import FIELD_NAMES, Book
ENCODING = "utf-8-sig"
```

여기에 대해 알려줘.

Claude 답변

줄	무엇	store.py에서 하는 일
<code>import csv</code>	CSV 읽기·쓰기	<code>DictReader</code> 로 읽고 <code>writer</code> 로 씀
<code>import os</code>	운영체제 기능	<code>os.replace()</code> 로 한 번에 교체
<code>import shutil</code>	파일 복사	<code>copy2()</code> 로 <code>.bak</code> 백업
<code>from pathlib import Path</code>	경로 클래스	경로 만들기, 파일 존재 확인
<code>from typing import NamedTuple</code>	이름 붙은 튜플	<code>LoadResult</code>

Path

```
Path("bookmate") / "books.csv" → bookmate/books.csv
p.name → books.csv / p.parent → bookmate / p.with_name("books.csv.bak") → bookmate/
books.csv.bak
```

/로 이르면 운영체제에 맞는 구분자로 바뀌 줍니다.

`from .models import ...`

점(.)은 "같은 패키지 안의" models입니다(상대 import). 그래서 이 파일을 직접 실행하면 에러가 납니다.

```
python bookmate/store.py → ImportError: attempted relative import with no known parent
package
python -c "import bookmate.store" → 성공
```

ENCODING = "utf-8-sig"

대문자 이름은 바꾸지 않는 상수라는 약속입니다. `utf-8-sig`는 파일 앞에 BOM(3바이트)을 붙이는 UTF-8입니다.

```
utf-8      : b'\xec\x80\x9c...'
utf-8-sig  : b'\xef\xbb\xbf\xec\x80\x9c...' ← 앞 3바이트가 BOM
```

엑셀은 BOM이 없으면 CP949로 읽어 한글이 깨집니다. 읽을 때도 `utf-8-sig`를 써야 BOM이 `'\ufeff'`처럼 글자로 남지 않습니다.

책 참고 4장(인코딩과 BOM)

Q18 · 나의 질문

```
class StoreError(Exception): ...
class LoadResult(NamedTuple): ...
class CsvStore: __init__, backup_path, temp_path, load, _backup_and_reset,
save
```

여기에 대해 알려줘.

Claude 답변

StoreError: 직접 만든 오류 종류

`Exception`을 상속하기만 하면 `raise` · `except`로 쓸 수 있는 오류가 됩니다. 몸통의 따옴표 세 개로 쓴 설명 글 (docstring)이 몸통 역할을 합니다. 저장 실패 원인이 여러 가지여도 이것 하나로 바꿔 올리면 화면은 `except StoreError` 한 줄이면 됩니다.

LoadResult: 읽기 결과 묶음

필드	뜻
books	읽어 온 책 목록
skipped	건너뛴 줄 수 (기본 0)
backup	<code>.bak</code> 백업 여부 (기본 False)

```
__init__: self.path = Path(path)
```

글자로 넘겨도 Path로 넘겨도 Path가 됩니다.

@property: 메서드를 값처럼

`store.backup_path`처럼 괄호 없이 읽으면 계산된 경로(`books.csv.bak`)가 나옵니다. 읽을 때마다 새로 계산해 항상 맞는 값입니다.

load(): 나가는 문이 다섯 개

파일 없음 → 빈 목록
 열기 실패(인코딩) → 백업 후 빈 목록
 내용 비었음 → 빈 목록
 헤더 다름 → 백업 후 빈 목록
 정상 → 한 줄씩 책으로 (실패한 줄은 건너뛴)

`tuple(...)`을 빼면 안 됩니다. `fieldnames`는 리스트, `FIELD_NAMES`는 튜플이라

```
['a', 'b'] == ('a', 'b') → False
tuple(['a', 'b']) == ('a', 'b') → True
```

`try`가 `for` 안에 있어서 한 줄이 실패해도 다음 줄을 계속 읽습니다.

`_backup_and_reset()`

`shutil.copy2`로 원본을 `.bak`으로 복사합니다. 빈 목록으로 시작한 뒤 종료 저장시 원본을 덮어쓰기 전에 사본을 남기기 위해서입니다.

책 참고 18장(with), 11장·22장(@property), 14장(예외 클래스 상속)

Q19 · 나의 질문

@property를 어떻게 사용하는지 예시랑 원리를 좀 더 설명해 주고 코드에서의 예시도 들어줘. 백업과 temp에 관한 지식이 많지 않아서 설명해 줘. 파일이 없는 걸 확인할 때 경로.exists로 확인할 수 있는 거니? with를 써서 파일이 닫힌다는 게 어떤 의미니? DictReader가 어디 있었지? if tuple부터 이후 내용을 더 자세히 알려 줘. 마지막으로 save 부분을 엮어서 자세히 설명해 줘.

Claude 답변

1. @property

원리: @property를 붙인 메서드는 괄호 없이 이름만 읽어도 파이썬이 그 메서드를 대신 실행해 결과값을 줍니다.

```
class 사각형:
    def __init__(self, w, h):
        self.w = w; self.h = h
    def 넓이_메서드(self):
        return self.w * self.h
    @property
    def 넓이(self):
        print("(넓이 계산 중)")
        return self.w * self.h
```

```
s.넓이_메서드    → <bound method ...>    ← 괄호 없으면 메서드 자체
s.넓이_메서드() → 12
s.넓이          → (넓이 계산 중) 12        ← 괄호 없이 읽었는데 실행됨
s.w = 10 후 s.넓이 → 40                    ← 항상 최신 값
s.넓이 = 99      → AttributeError          ← 함부로 바꿀 수 없음
```

BookMate의 예: `store.backup_path`, `store.temp_path`, `dialog.goal` (통계 창 의 숫자 칸 값). 모두 다른 값을 보고 그때그때 계산하는 정보입니다.

2. 백업(.bak)과 임시(.tmp)

파일	역할	생기는 때	사라지는 때
books.csv	진짜 데이터	처음 저장	사라지지 않음
books.csv.bak	비상용 사본	읽을 때 파일이 이상하면	자동으로 안 지워짐
books.csv.tmp	작업용 종이	저장할 때마다	저장 끝나는 즉시

3. exists()

네. 있으면 True, 없으면 False이고 폴더에도 쓸 수 있습니다. 처음 실행엔 파일이 없는 게 정상이라 빈 목록으로 시작합니다.

4. with로 파일이 닫힌다는 뜻

열린 파일은 운영체제에 **빌린 상태**입니다. 쓴 내용은 잠시 모아 두었다가 한꺼번에 들어갑니다.

```
with 안 (3,000바이트 쓴 직후): 닫혔나? False / 디스크 크기: 0
with 밖:                        닫혔나? True / 디스크 크기: 3000
```

닫기는 모아 둔 내용을 마저 쓰고 파일을 돌려주는 것입니다. Windows는 열린 파일을 옮기지 못하게 합니다. **with**는 정상이든 **return**이든 오류든 블록을 나가면 **무조건** 닫습니다. (**try/finally**로 **close()**를 부르는 것을 짧게 쓴 것)

5. DictReader

표준 라이브러리 **csv 모듈**에 있고, store.py 맨 위 **import csv**로 가져와 **load()**에서 씁니다.

6. if tuple 이후, 한 줄씩

예시 파일 (2번째 책은 연도가 "미상"):

```
title,author,year,genre,isbn,rating,read,favorite
코스모스,칼 세이건,2006,과학,1-1,4.5,True,True
데미안,헤르만 헤세,미상,소설,1-2,,False,False
아몬드,손원평,2017,소설,1-3,4.0,True,False
```

A. 헤더 확인: 리스트를 **tuple()**로 바꿔 비교 → 같으면 통과 B. 그릇 준비: **books = []**, **skipped = 0** C. 한 줄씩:

```
1번째 줄 코스모스 year='2006' → 성공          | books 1권, skipped 0
2번째 줄 데미안  year='미상' → 실패 → 건너뛴   | books 1권, skipped 1
3번째 줄 아몬드  year='2017' → 성공          | books 2권, skipped 1
```

D. **return LoadResult(books, skipped)** → 책 2권, 건너뛴 1

7. save()를 위하여

```
① tmp = self.temp_path          → 이름만 정함   폴더: [books.csv]
② with open(tmp, "w", ...)      → 임시 파일 생김
③ writerow(FIELD_NAMES), writerows(...to_row()...) → 헤더와 책들 씀
④ with 끝 → 파일 닫힘          폴더: [books.csv, books.csv.tmp]
⑤ os.replace(tmp, self.path)    → 한 번에 교체   폴더: [books.csv]
⑥ 실패하면 tmp 지우고 StoreError
```

④에서 닫혀 있어야 ⑤가 가능합니다. load와 save는 짝입니다.

```
books.csv —DictReader→ from_row → Book 목록
books.csv ←os.replace← .tmp ← csv.writer ← to_row
```

책 참고 11장·22장(@property), 18장(with)

Q20 · 나의 질문

임시 파일은 파일을 생성하기 전에 만든다고 이해했는데, 백업 경로는 어디서 사용되는지 잘 모르겠어.

Claude 답변

임시 파일

정확한 순서: 저장할 때 `books.csv.tmp` 를 새로 만들어 끝까지 쓰고 → `os.replace()` 로 기존 `books.csv` 자리에 갈아 끼웁니다. "books.csv를 만들기 전에"가 아니라 "**books.csv를 새 내용으로 바꾸기 전에**" 옆에서 미리 다 써 두는 파일입니다.

백업 경로가 쓰이는 곳

코드 전체에서 **딱 한 곳**, `_backup_and_reset()` 입니다. 이 메서드는 `load()` 의 두 곳(헤더가 다를 때, 글자를 읽을 수 없을 때)에서 불립니다.

	temp_path	backup_path
쓰이는 메서드	<code>save()</code>	<code>load()</code> 안의 <code>_backup_and_reset()</code>
언제	저장할 때마다	파일이 이상할 때만

실제로 헤더가 엉뚱한 파일로 컷다 끈 결과:

- ① 시작: `['books.csv']`
- ② load 후: `['books.csv', 'books.csv.bak']` 책 0권, `backup = True`
- ③ 종료 저장 후: `books.csv` = 헤더만 / `books.csv.bak` = 원래 내용 그대로

백업이 없었다면 ③에서 원래 기록이 영영 사라졌을 겁니다.

5부. manager.py

Q21 · 나의 질문

manager.py 전체(Result, Stats, _normalize, BookManager의 load·save·add_book·update_book·delete_book·_make_book·list_all·search·get_stats·toggle_favorite)에 대해서 알려줘.

Claude 답변

먼저: 예전 버전 주의

`eul_reul()` 함수가 빠진 예전 코드라면 테스트와 `main_window.py`가 `ImportError`를 냅니다. `_normalize` 위에 추가하고, 메시지 세 줄을 `f'{{book.title}}'{{eul_reul(book.title)}} 등록했습니다` 형태로 바꿉니다.

```
def eul_reul(word: str) -> str:
    if word and "가" <= word[-1] <= "힉":
        return "을" if (ord(word[-1]) - ord("가")) % 28 else "를"
    return "을(를)"
```

Result와 default_factory

```
errors: list = [] → ValueError: mutable default ... is not allowed: use default_factory
```

`= []` 는 클래스를 만들 때 한 번만 만들어져 모든 Result가 **같은 리스트를 공유**합니다. `field(default_factory=list)` 는 만들 때마다 새 리스트를 만듭니다.

```
a.errors에 "오류" 추가 → a.errors = ['오류'] / b.errors = []
```

_normalize(): NFC + casefold

보이는 글자: 한강 / 글자 수: 6 vs 2 / == : False ← 자모 분리형(NFD)
 NFC로 정규화 후 == : True
 "Clean CODE".casefold() → "clean code"

__init__

`store` 를 밖에서 받아 테스트에서 임시 파일을 넣을 수 있습니다(의존성 주입). `validator or Validator()` 는 넘겨받은 게 없으면 새로 만듭니다.

load()

```
self.books.append(book)          # 리스트에 그 책을
self.isbn_map[book.isbn] = book  # 딕셔너리에도 "같은 그 책"을
```

`continue` 는 이번 바퀴를 건너뛰고 다음으로(같은 ISBN이 또 나오면). `_replace()` 는 일부 값만 바꾼 새 묶음.

add_book() / update_book() / delete_book()

- 등록: `set(self.isbn_map)` 으로 ISBN 집합을 만들어 검증 → 통과하면 양쪽에 넣음
- 수정: `{**data, "isbn": isbn}` 으로 ISBN 고정. 새 책을 넣지 않고 **원래 책의 내용만 바꿔** 양쪽에 자동 반영
- 삭제: `pop(isbn, None)` 으로 딕셔너리에서 꺼내며 지우고 `remove` 로 리스트에서도 지움

```
{**data, "isbn": "978-1"} → {"title": "코스모스", "isbn": "978-1"}    (data는 그대로)
m.pop("a", None) → 1 / m.pop("z", None) → None
```

@staticmethod (_make_book)

데코레이터	첫 매개변수	언제
없음	self	객체의 값을 써야 할 때
@classmethod	cls	클래스로 새 객체를 만들 때
@staticmethod	없음	받은 값만으로 일할 때

list_all / search

`list(self.books)` 는 리스트만 복사(안의 책은 같은 책 = 얇은 복사). `getattr(b, "author")` 는 `b.author` 와 같아, 검색할 칸이 글자로 와도 한 줄로 처리합니다.

get_stats

`max(goal, 1)` 로 0 나누기 방지, `sum(1 for b in books if b.read)` 로 개수 세기, 별점 있는 책만 평균(`round(..., 1)`), 하나도 없으면 None.

toggle_favorite

`[isbn]` 으로 일부러 `KeyError` 가 나게 하고(반환값이 bool이라 실패를 표현할 자리가 없음), 화면이 잡아서 안내합니다.
`not` 으로 `True↔False` 를 뒤집습니다.

책 참고 5장(default_factory), 6장(공유·얇은 복사), 3장(`{**data}`), 11장(@staticmethod)

6부. 파일 처리와 안전 저장

Q22 · 나의 질문

잘못된 csv 파일을 넣으면 그냥 아예 나오지 않는 거니?

Claude 답변

얼마나 잘못됐느냐에 따라 다릅니다.

경우	목록	안내	원본
① 일부 행만 잘못됨 (연도 "미상", 칸 부족, ISBN 중복 등)	정상 행은 전부 나옴	"N개 행을 건너뛰었습니다"	그대로
② 헤더가 다름	빈 목록	"백업하고 빈 목록으로 시작"	.bak 으로 복사
③ UTF-8이 아님 (CP949 등)	빈 목록	②와 같음	.bak 으로 복사
④ 파일이 없거나 비어 있음	빈 목록	없음	—

②③은 어느 칸이 무엇인지조차 알 수 없어서 한 줄도 믿지 않고 백업 후 시작합니다.

주의 1: ②③ 상태로 창을 닫으면 books.csv는 빈 목록으로 덮어써지지만 원본은 .bak 에 남습니다. .bak 은 하나뿐이라 복구 전에 또 잘못된 파일로 실행하면 이전 백업이 덮어써집니다.

주의 2: 형식은 맞지만 값이 규칙 밖인 줄(연도 3000, 별점 7.0)은 그대로 들어옵니다. 적재할 때는 "숫자로 바꿀 수 있는가"만 보고 Validator 규칙은 적용하지 않기 때문입니다.

Q23 · 나의 질문

(저장 실패 캡처와 save() 코드를 보여 주며) 이 이미지면 되니? 그리고 내가 첨부한 코드가 실제 기능을 수행하는 게 맞니? 맞다면 원리를 좀 더 자세히 알려줘.

Claude 답변

맞습니다. 다만 두 곳이 나눠서 일합니다. save() 는 실패를 감지하고 원본을 지키고, 안내창은 main_window.py 의 closeEvent() 가 띄웁니다.

```
X 클릭
└─ Qt가 MainWindow.closeEvent() 호출
    └─ BookManager.save()
        └─ CsvStore.save(books) ← 첨부한 코드
            └─ os.replace() 실패 → StoreError 발생
                └─ except StoreError: QMessageBox.warning(...) ← 캡처의 안내창
                    └─ event.accept() → 창 닫힘
```

엑셀이 books.csv를 연 상태에서 한 줄씩

1. tmp = self.tmp_path — 다른 이름(books.csv.tmp)이라 엑셀과 무관
2. with open(tmp, "w", ...) 와 세 줄 — 임시 파일에 다 씀. **여기까지 성공**
3. with 끝 — 파일이 닫힘. 열린 파일은 Windows가 옮기지 못하게 하므로 반드시 필요
4. os.replace(tmp, self.path) — **여기서 실패**. 엑셀이 "다른 프로그램은 이 파일을 지우거나 바꾸지 마라"는 조건으로 열려 뒤서 Windows가 거부 → **PermissionError**. 거부는 이름표에 손대기 전에 일어나므로 books.csv는 그대로
5. except OSError — PermissionError는 OSError의 자식이라 잡힘

6. `tmp.unlink(missing_ok=True)` — 쓰다 만 임시 파일 삭제
7. `raise StoreError(...) from exc` — 우리 프로그램의 말로 바꿔 위로

이번 실행에서 바꾼 내용은 저장되지 않습니다(캡처에서 9권이었던 재실행 시 원본대로 8권).

날카로운 질문 대비

"엑셀 잠금이면 직접 쓰는 방식도 `open()` 에서 실패하지 않나?" → **맞습니다**. 임시 파일 방식이 진짜 필요한 건 **쓰기 시작한 뒤 실패하는 경우**(디스크 가득 참, 강제 종료, 정전)입니다. "`w`" 로 여는 순간 원본이 지워지기 때문입니다. 엑셀 시연은 "실패 시 원본을 지키고 알려 주는 흐름"을 보여 주는 것입니다.

책 참고 18장(with)

Q24 · 나의 질문

(앞의 설명을 붙여 넣으며) 여기에 대해 좀 더 자세하게 설명해 줘.

Claude 답변

먼저 바로잡을 두 가지

1. "with가 끝나면 디스크로 다 내려간다"는 부정확합니다. `close()` 는 **운영체제에 넘기는 것**까지만 보장하고, 물리적 디스크 기록은 `os.fsync()` 가 필요합니다.
2. 원자적 교체는 리눅스·맥은 표준으로 보장, Windows는 같은 NTFS 드라이브 안에서 실제로는 한 번에 처리되지만 공식 문서가 명시하지는 않습니다.

배경 1: 파일은 "내용"과 "이름"이 따로

`os.replace()` 는 내용을 복사하지 않고 **books.csv**라는 이름표를 새 내용으로 옮겨 붙입니다.

교체 전 tmp 파일의 고유번호 : 2801682

교체 후 books.csv의 고유번호: 2801682 ← 같은 파일에 이름만 바뀜

배경 2: "w"로 여는 순간 지워진다

열기 전 크기: 70 바이트

열자마자 크기: 0 바이트 ← 아직 한 글자도 안 썼는데 0

배경 3: 쓴 내용의 세 단계

`write()` → [파이썬 버퍼] → [운영체제 캐시] → [실제 디스크]

`close()`가 `os.fsync()`가

여기까지 보장 여기까지 보장

`write()` 직후 디스크상 크기: 0 / `close()` 직후: 3000

실험: 2,000줄을 쓰다가 1,500번째 줄에서 실패

방식	실패 후 books.csv
A. 바로 쓰기	1,500줄만 남음, 원래 책은 사라짐
B. 임시 파일 후 교체	원본 그대로

단계별 보충

- 임시 파일을 같은 폴더에 두는 이유: 이름표 옮기기는 같은 드라이브 안에서만 가능
- Windows 공유 모드: 엑셀은 "삭제 공유"를 허락하지 않음 → replace는 옛 이름표를 떼야 하므로 거절(보통 WinError 32)
- `except OSError`로 좁게 잡는 이유: 코드 버그(TypeError 등)까지 "저장 실패"로 덮지 않기 위해
- `from exc`: 트레이스백에 "위 예외가 아래 예외의 직접 원인"이라는 연결이 남음

이 코드가 막지 못하는 경우

상황	결과	개선
쓰는 도중 OSError가 아닌 오류	임시 파일이 남고 오류가 올라감 (원본은 안전)	정리를 <code>finally</code> 로
교체 직후 정전	캐시의 새 내용이 디스크에 못 갔을 수 있음	<code>f.flush() + os.fsync()</code>

Q25 · 나의 질문

(save() 코드를 다시 붙여 넣으며) 좀 쉽게 설명해 줘.

Claude 답변

비유: 칠판에 적힌 도서 목록 고치기

나쁜 방법: 칠판을 먼저 싹 지우고 새로 적기 시작. 적다가 종이 울려 멈추면 반만 적힌 목록만 남고 원래 목록은 없음.

좋은 방법: 칠판은 그대로 두고 옆 종이에 새 목록을 끝까지 다 적은 다음 칠판에 탁 붙임. 중간에 멈추면 종이만 버리면 됨.

- 칠판 = `books.csv` / 옆의 종이 = `books.csv.tmp` / 탁 붙이기 = `os.replace()`

한 줄씩

코드	쉬운 말
<code>tmp = self.temp_path</code>	옆에 쓸 종이 준비
<code>try:</code>	실패할 수도 있으니 조심해서
<code>with open(tmp, "w", ...) as f:</code>	종이를 쓰기용으로 열고, 다 쓰면 알아서 닫아 줘
<code>writer.writerow(FIELD_NAMES)</code>	맨 윗줄 제목 쓰기
<code>writer.writerows(...)</code>	책을 한 권씩 한 줄로
<code>os.replace(tmp, self.path)</code>	다 적은 종이를 칠판에 한 번에 붙이기
<code>except OSError as exc:</code>	파일 관련 문제가 생기면
<code>tmp.unlink(missing_ok=True)</code>	쓰다 만 종이 버리기 (없어도 괜찮음)
<code>raise StoreError(...) from exc</code>	화면에 "저장 실패"라고 알리기

한 문장 요약

원본은 그대로 두고 새 파일을 옆에 다 만든 다음, 한 번에 바꿔 끼운다. 그래서 중간에 실패해도 원본은 안전하다.